

CAHIER DES CHARGES TECHNIQUE

Plateforme Smart University

Projet d'Ingénierie Académique

Projet	Plateforme Smart University
Version	1.0
Date	Février 2025
Échéance	20 mai 2025
Équipe	3 Étudiants en Ingénierie
Statut	En cours de réalisation

1. Introduction

1.1 Contexte du Projet

La Plateforme Smart University est un projet d'ingénierie académique développé par une équipe de trois étudiants. L'objectif est de moderniser l'infrastructure d'un campus universitaire en intégrant des technologies de l'Internet des Objets (IoT), du cloud computing et de l'intelligence artificielle. Le système vise à offrir une gestion intelligente et en temps réel des espaces, des équipements et de la sécurité du campus.

La plateforme repose sur une architecture à trois couches : une couche Edge composée d'appareils IoT déployés dans les salles, une couche Plateforme hébergeant les services de communication et de persistance des données, et une couche Application constituée d'une application mobile Flutter et d'un module d'intelligence artificielle.

1.2 Objectifs du Projet

Le projet vise à atteindre quatre objectifs principaux :

- Mettre en place une architecture de communication event-driven entre les appareils IoT et l'application mobile, en utilisant le protocole MQTT pour le contrôle des actionneurs en temps réel.
- Déployer l'ensemble des services backend sur un serveur dédié, garantissant la disponibilité continue du système.
- Intégrer un module d'intelligence artificielle capable de détecter l'occupation des salles et d'alerter en cas d'intrusion hors des heures d'ouverture, par vision par ordinateur.
- Sécuriser l'accès au flux vidéo en direct en implémentant une authentification par token sur le point de terminaison de streaming.

1.3 Motivation Stratégique

Du point de vue des performances, l'utilisation du protocole MQTT en mode publish/subscribe permet de réduire la latence de réponse des actionneurs à moins de deux cents millisecondes, contre plusieurs secondes avec un mécanisme de polling classique. Du point de vue de la scalabilité, l'architecture à couches établit une séparation claire des responsabilités, permettant un dimensionnement indépendant de chaque composant. Du point de vue de l'intelligence, l'intégration d'un module de vision par ordinateur transforme le système en un gestionnaire d'environnement actif, capable de détecter les accès non autorisés et de surveiller l'occupation des salles en temps réel.

2. Périmètre du Projet

2.1 Fonctionnalités incluses

Les composants et fonctionnalités suivants sont dans le périmètre du projet et constituent des livrables engagés pour l'échéance du 20 mai 2025 :

- Développement du firmware IoT pour les appareils ESP32 et ESP8266 avec communication MQTT en publish/subscribe pour la publication des données capteurs et le contrôle des actionneurs.
- Déploiement d'un broker MQTT Mosquitto sur un serveur pour assurer la communication entre les appareils IoT et le serveur pont.

- Développement d'un serveur pont Node.js assurant le pontage entre Supabase Realtime et MQTT, l'ingestion de la télémétrie des capteurs, l'enregistrement des présences RFID et le relais du flux vidéo authentifié.
- Développement d'un module d'inférence IA basé sur YOLOv8n pour la détection de personnes en temps réel, le comptage d'occupation par salle et l'envoi d'alertes d'intrusion hors heures.
- Développement d'une application mobile Flutter incluant un tableau de bord d'occupation, des notifications d'alerte en temps réel, l'accès au flux vidéo sécurisé et une interface de contrôle des actionneurs.
- Conception et déploiement du schéma de base de données Supabase avec les politiques de Row Level Security (RLS) appropriées.

3. Exigences Fonctionnelles

Les exigences fonctionnelles sont classées par priorité selon la méthode MoSCoW : Must Have (M — Indispensable), Should Have (S — Important), Could Have (C — Souhaitable). Chaque exigence est identifiée par un code unique au format FR-XX.

3.1 Couche IoT

ID	Exigence	Priorité	Acteur
FR-01	Publication des données capteurs via MQTT	Must Have	ESP32 / ESP8266
Description :	Les appareils ESP32 et ESP8266 doivent publier les lectures des capteurs — incluant température, humidité, concentration de gaz, état de détection de mouvement et intensité lumineuse — vers des topics MQTT structurés à des intervalles configurables ne dépassant pas trente secondes.		

FR-02	Contrôle des actionneurs en temps réel via MQTT	Must Have	ESP32 / ESP8266
Description :	Les appareils doivent s'abonner à leur topic MQTT d'actionneurs désigné et exécuter les commandes dans les deux cents millisecondes suivant la réception du message.		

FR-03	Publication des événements de présence RFID	Must Have	ESP32 / RFID-RC522
Description :	Lors de la détection d'un tag RFID valide, le firmware ESP32 doit publier un événement de présence vers le topic MQTT désigné, incluant l'identifiant du tag et un horodatage UTC.		

FR-04	Rapport d'état de connectivité des appareils	Should Have	ESP32 / ESP8266
Description :	Les appareils doivent publier un message heartbeat vers un topic de statut à des intervalles de soixante secondes. L'absence de heartbeat pendant plus de quatre-vingt-dix secondes doit être signalée comme un événement appareil-hors-ligne.		

3.2 Module IA

ID	Exigence	Priorité	Acteur
FR-06	Détection de personnes en temps réel	Must Have	Serveur IA / YOLOv8n
Description :	Le module IA doit traiter les trames vidéo du flux ESP32-CAM à un taux minimum de cinq trames par seconde et détecter la présence de personnes en utilisant un modèle YOLOv8n avec un seuil de confiance d'au moins 0,50.		

FR-07	Persistance du comptage d'occupation	Must Have	Serveur IA / Supabase
Description :	Après chaque cycle d'inférence, le module IA doit écrire le nombre de personnes détectées pour la salle surveillée dans la table room_occupancy de Supabase, incluant l'identifiant de la salle et l'horodatage UTC.		

FR-08	Détection d'intrusion hors heures	Must Have	Serveur IA / Supabase
Description :	Lorsqu'une personne est détectée en dehors des heures opérationnelles — définies comme avant 07h00 et après 22h00 heure locale — le module IA doit insérer un enregistrement d'alerte de type intrusion dans la table alerts de Supabase.		

FR-09	Heures opérationnelles configurables	Should Have	Serveur IA
Description :	Les limites des heures opérationnelles doivent être configurables via une variable d'environnement ou un fichier de configuration, sans nécessiter de modification du code d'inférence.		

3.3 Couche Backend et Communication

ID	Exigence	Priorité	Acteur
FR-11	Déploiement d'un broker MQTT persistant	Must Have	Serveur / Mosquitto
Description :	Un broker MQTT Mosquitto doit être déployé sur un serveur et rester disponible en continu, acceptant les connexions des appareils IoT et du serveur pont Node.js sur le port 1883.		

FR-12	Pont Supabase Realtime vers MQTT	Must Have	Node.js / Supabase / MQTT
Description :	Le serveur Node.js doit s'abonner aux événements de changement Supabase Realtime sur la table actuators. À la réception d'un événement, le serveur doit publier la commande correspondante vers le topic MQTT de l'appareil approprié dans les cinq cents millisecondes.		

FR-13	Ingestion de télémétrie MQTT vers Supabase	Must Have	Node.js / Supabase
Description :	Le serveur Node.js doit s'abonner à tous les topics MQTT de données de capteurs et persister les charges utiles reçues dans les tables Supabase appropriées, en enrichissant chaque enregistrement d'un horodatage UTC côté serveur.		

FR-14	Enregistrement des présences RFID	Must Have	Node.js / Supabase
Description :	Les événements de présence reçus via MQTT doivent être persistés dans la table attendance de Supabase, incluant l'identifiant du tag, l'identité inférée de l'étudiant ou du personnel, l'identifiant de la salle et l'horodatage.		

FR-15	Point de terminaison de diffusion vidéo authentifié	Must Have	Node.js / Supabase Auth
Description :	Le point de terminaison WebSocket de streaming Node.js doit valider un token de session annexé à l'URL de connexion auprès du service d'authentification Supabase. Les connexions non authentifiées doivent être rejetées avec le statut HTTP 401.		

3.4 Application Mobile

ID	Exigence	Priorité	Acteur
FR-17	Tableau de bord d'occupation en temps réel	Must Have	Flutter / Supabase Realtime
Description :	L'application Flutter doit inclure un écran dédié affichant les comptages de personnes en temps réel pour chaque salle surveillée, via des abonnements Supabase Realtime sur la table room_occupancy. L'affichage doit se mettre à jour automatiquement, sans actualisation manuelle.		

FR-18	Notifications d'alerte en temps réel	Must Have	Flutter / Supabase Realtime
Description :	Lors de l'insertion d'un nouvel enregistrement dans la table alerts de Supabase, l'application Flutter doit afficher une notification in-app dans les deux secondes, quel que soit l'écran actuellement actif.		

FR-19	Accès au flux vidéo authentifié	Must Have	Flutter / Supabase Auth
Description :	L'application Flutter doit récupérer le token de session de l'utilisateur authentifié depuis Supabase et l'annexer comme paramètre URL lors de l'initiation d'une connexion WebSocket au point de terminaison de streaming.		

FR-20	Interface de contrôle des actionneurs	Must Have	Flutter / Supabase
--------------	--	-----------	--------------------

Description :	L'application Flutter doit fournir une interface permettant aux utilisateurs autorisés d'envoyer des commandes aux actionneurs individuels — servo, buzzer, LED RGB et relais ventilateur — via la table actuators de Supabase.
----------------------	---

4. Exigences Non-Fonctionnelles

ID	Catégorie	Exigence	Cible
NFR-01	Performance	La latence bout en bout de la commande d'un actionneur — de l'action utilisateur dans l'application mobile jusqu'à l'exécution par l'appareil — ne doit pas dépasser cinq cents millisecondes dans des conditions réseau normales.	< 500 ms
NFR-02	Performance	Le module d'inférence IA doit traiter chaque trame vidéo et publier le résultat d'occupation dans la seconde suivant la capture de la trame.	< 1 s par trame
NFR-03	Performance	Les notifications d'alerte doivent apparaître dans l'application Flutter dans les deux secondes suivant l'insertion de l'événement dans la table alerts de Supabase.	< 2 s
NFR-05	Sécurité	Le point de terminaison WebSocket de diffusion vidéo doit rejeter toutes les connexions ne présentant pas un token de session Supabase valide. L'authentification doit être vérifiée avant toute transmission de données.	Validation de token requise
NFR-06	Sécurité	Les politiques de Row Level Security (RLS) de Supabase doivent être appliquées sur toutes les tables. Les étudiants disposent d'un accès en lecture seule à leurs propres enregistrements de présence. Le personnel dispose d'un accès en lecture aux données de capteurs des salles de ses bâtiments.	RLS appliqué
NFR-7	Maintenabilité	Tout le code source doit être maintenu dans un dépôt Git versionné avec des messages de commit descriptifs organisés par composant. Un fichier README doit documenter les procédures d'installation et de déploiement de chaque composant.	Git, documenté
NFR-11	Maintenabilité	Le serveur Node.js doit être structuré avec des modules logiquement séparés pour la gestion MQTT, l'intégration Supabase et le relais de flux vidéo, afin de permettre la modification indépendante de chaque préoccupation.	Structure modulaire

5. Architecture Technique

5.1 Couches du Système

L'architecture de la plateforme est organisée en trois couches distinctes, reflétant la séparation des responsabilités entre les différents composants du système.

Couche 1 — Couche Edge (Terrain)

La Couche Edge regroupe l'ensemble du matériel physique déployé dans les salles du campus. Les microcontrôleurs ESP32 assurent les fonctions IoT principales : lecture des capteurs MQ2 (gaz), HC-SR04 (ultrason), PIR (mouvement) et RFID-RC522 (présences) ; contrôle des actionneurs (servomoteur, buzzer, LED RGB, afficheur LCD) ; et streaming vidéo MJPEG via le module ESP32-CAM. Les microcontrôleurs ESP8266 prennent en charge la mesure environnementale via les capteurs DHT11 (température, humidité) et LDR (luminosité), ainsi que le contrôle d'un ventilateur par relais. Tous les appareils communiquent exclusivement via WiFi.

Couche 2 — Couche Plateforme

La Couche Plateforme regroupe les services hébergés sur un serveur et le backend Supabase. Un broker MQTT Mosquitto est déployé sur un serveur et accepte les connexions des appareils IoT sur le port 1883. Un serveur pont Node.js assure le rôle de passerelle architecturale : il s'abonne à tous les topics MQTT des appareils, persiste les données de capteurs et de présence dans Supabase, et transmet les commandes issues des événements Supabase Realtime vers les topics MQTT des appareils. Ce même processus opère le relais WebSocket du flux vidéo MJPEG, avec vérification d'authentification. Supabase fournit la base de données PostgreSQL managée, le service d'authentification utilisateur et la livraison d'événements Realtime.

Couche 3 — Couche Application

La Couche Application est constituée de l'application mobile Flutter et du module d'inférence IA. L'application Flutter communique avec Supabase via HTTPS REST pour les opérations de données, et via WebSocket Supabase Realtime pour les mises à jour en direct. Elle se connecte au flux vidéo via WebSocket en transmettant un token de session. Le module d'inférence IA est une application Python exécutée sur un serveur dédié ou un ordinateur portable lors des démonstrations : il lit les trames depuis le flux ESP32-CAM, effectue la détection de personnes avec YOLOv8n, et écrit les résultats directement dans Supabase via HTTPS.

5.2 Flux de Données

Flux des données capteurs : L'appareil publie vers un topic MQTT → Le serveur pont reçoit le message → Le pont persiste l'enregistrement dans la table de capteurs Supabase → L'application Flutter reçoit la mise à jour via Supabase Realtime → Le tableau de bord se met à jour.

Flux des commandes actionneurs : L'utilisateur interagit avec l'interface Flutter → Flutter écrit la commande dans la table actuators de Supabase → Supabase Realtime délivre l'événement au serveur pont → Le pont publie le message MQTT vers le topic de l'appareil → L'appareil exécute la commande.

Flux d'occupation IA : Le module IA capture une trame depuis le flux MJPEG → L'inférence YOLOv8n produit un comptage de personnes → Le module IA écrit le comptage dans la table room_occupancy → Si hors heures et comptage > 0, une alerte est insérée dans la table alerts → L'application Flutter reçoit l'alerte via Supabase Realtime → Notification affichée à l'utilisateur.

Flux de présence RFID : L'ESP32 détecte un tag RFID → Le firmware publie l'événement vers MQTT → Le serveur pont reçoit l'événement → Le pont persiste l'enregistrement dans la table attendance de Supabase → L'application Flutter affiche la confirmation.

5.3 Protocoles de Communication

Protocole	Transport	Cas d'Utilisation	Port
MQTT 3.1.1	TCP	Publication des capteurs, abonnement aux actionneurs	1883
HTTPS / REST	TLS sur TCP	Lecture/écriture Supabase, écritures du module IA	443
WebSocket (Supabase Realtime)	TLS sur TCP	Abonnements temps réel, événements de changement actionneurs	443
WebSocket (Relais MJPEG)	TCP	Ingestion flux ESP32-CAM et diffusion vers les clients	8080 / 3000
WiFi 802.11 b/g/n	Couche 2	Connectivité réseau de tous les appareils	—

6. Exigences Matérielles et Logicielles

6.1 Matériel

Composant	Quantité	Rôle
Carte de développement ESP32	1 par salle surveillée	Nœud IoT principal (capteurs + actionneurs)
Carte de développement ESP8266	1 par salle surveillée	Nœud de détection environnementale
Module ESP32-CAM	1 par salle surveillée	Source de streaming vidéo pour le module IA
Capteur de Gaz/Fumée MQ2	1 par ESP32	Détection de gaz et de fumée
Capteur Ultrasonique HC-SR04	1 par ESP32	Détection de distance / proximité
Capteur de Mouvement PIR	1 par ESP32	Détection de mouvement infrarouge passif
Module RFID-RC522	1 par ESP32	Lecture des tags de présence
Capteur Température/Humidité DHT11	1 par ESP8266	Acquisition de données environnementales
Capteur de Lumière LDR	1 par ESP8266	Mesure de la luminosité ambiante
Servomoteur (SG90)	1–2 par nœud	Actionneur de porte / ventilation
Module Relais (5V)	1 par ESP8266	Contrôle du ventilateur

Composant	Quantité	Rôle
LED RGB + Buzzer	1 par ESP32	Indication d'alerte visuelle et sonore
Afficheur LCD I2C (16x2)	1 par ESP32	Affichage de statut local
PC Portable / Serveur	1	Serveur d'inférence IA lors des démonstrations

6.2 Stack Logicielle

Composant	Technologie	Version / Notes
Application Mobile	Flutter (Dart)	Flutter 3.x, package supabase_flutter
Firmware IoT	Framework Arduino / PlatformIO	PubSubClient (MQTT), ArduinoJson
Serveur Pont	Node.js	Node.js 20 LTS, mqtt.js, @supabase/supabase-js
Broker MQTT	Eclipse Mosquitto	v2.x
Inférence IA	Python 3.11, Ultralytics YOLOv8n, OpenCV	YOLOv8n.pt (~6 Mo), supabase-py
Backend / Base de Données	Supabase (PostgreSQL managé)	Realtime activé, RLS appliqué
Contrôle de Version	Git / GitHub	Un dépôt par composant majeur
Outils de Diagramme	draw.io	Diagrammes d'architecture, flux de données et ER